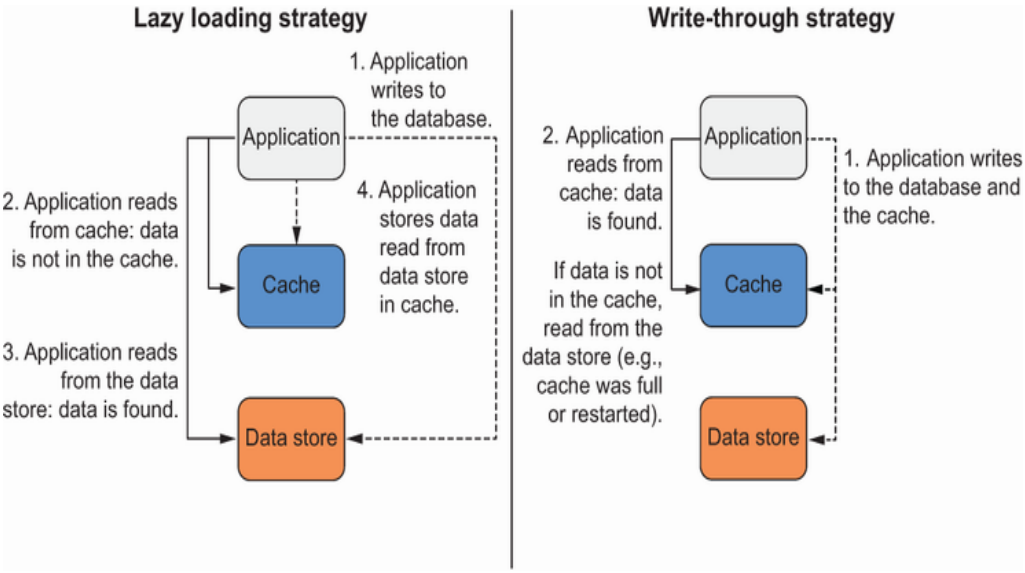


Figure 11.2 Comparing the lazy-loading and write-through caching strategies

When evicting data, the cache needs to decide which data it should delete. One popular strategy is to evict the least recently used (LRU) data. This means that cached data must contain meta information about the time when it was last accessed. In case of an LRU eviction, the data with the oldest timestamp is chosen for eviction.

Caches are usually implemented using key-value stores. Key-value stores don't support sophisticated query languages such as SQL. They support retrieving data based on a key, usually a string, or specialized commands, for example, to extract sorted data efficiently.

Imagine that in your relational database is a `player` table for your mobile game. One of the most common queries is `SELECT id, nickname FROM player ORDER BY score DESC LIMIT 10` to retrieve the top 10 players. Luckily, the game is very popular, but this comes with a technical challenge. If many players look at the leaderboard, the database becomes very busy, which causes high latency or even timeouts. You have to come up with a plan to reduce the load on the database. As you have already learned, caching can help. What technique should you employ for caching? You have a few options.

One approach you can take with Redis is to store the result of your SQL query as a string value and the SQL statement as your key name. Instead of using the whole SQL query as the key, you can hash the string with a hash function like `md5` or